# AI651 – Deep Learning for Space, Time and Graphs

Assignment 2

Name: **Imran Saeed** Roll number: **25280083**

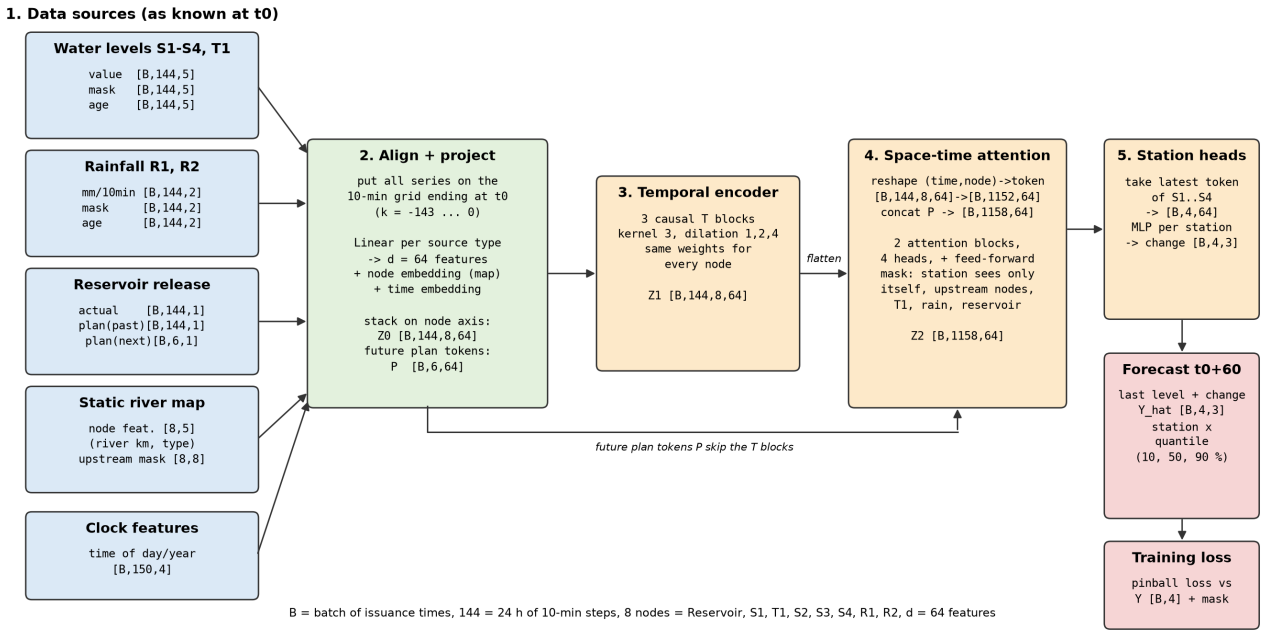
GitHub repository (public): `https://github.com/Imran-TheProgrammer/space-time-forecasting-convlstm-and-attention`

---

## Question 1 – River Forecasting Proposal

**Summary of the design.** Every 10 minutes we build a 24-hour history for 8 places: the reservoir, S1, T1, S2, S3, S4, R1 and R2. A small time encoder cleans each place's history. Then one attention step lets each station look at its own past, all places upstream of it, the rain gauges and the announced future releases, *at any time lag*. A small head per station predicts the change in water level over the next hour, as three quantiles (10%, 50%, 90%).

### 1. Architecture diagram




**Q1 architecture.** $B$ = batch of issuance times. $144 = 24$ h of 10-minute steps. Node order: Reservoir, S1, T1, S2, S3, S4, R1, R2. All inputs use only data that has arrived by the issuance time $t_0$.

**Joining the sources.** (1) Every series is put on one 10-minute grid that ends at $t_0$. (2) Each source type has its own linear layer that maps its features to the same size $d = 64$. (3) The results are stacked on a node axis: $[B, 144, 8, 64]$. (4) Before attention, the time and node axes are flattened into one token axis: $[B, 1152, 64]$. The 6 future release tokens are added at the end: $[B, 1158, 64]$. (5) The heads read the token at time $t_0$ for S1–S4: $[B, 4, 64]$.

### 2. Assumptions

**Supplied facts (not assumptions):** levels at S1–S4 and T1 every 10 min; rain at R1 and R2 with timestamps and locations; recorded releases and announced schedules with announcement and effective times; station locations, river connections and distances; no data on the unmonitored tributary; no future rain or future levels at issuance.

| No. | Assumption and why it is reasonable | Effect on the design |
|---|---|---|
| A1 | All clocks use one time zone (UTC), and each level timestamp marks the end of its 10-minute interval. This is normal for a single authority's network. | Readings are placed on the grid by timestamp, with no shifting. |
| A2 | Level readings usually arrive within 10–20 minutes, but sometimes much later. The arrival time of each reading is logged. | We add an "age" input. Training uses only data that had arrived by each past $t_0$ ("as-of" data). |
| A3 | Rain gauges report at uneven times (for example tipping buckets). | Rain is summed into 10-minute totals on the same grid. |
| A4 | The record covers at least 3–5 years at 10-minute steps, including several floods. | This is enough data to learn travel times and rain response. Train, validation and test are split by whole time periods, not at random. |
| A5 | Level sensors have small noise (about 1 cm) and sometimes spikes or flat lines. The authority flags known bad data. | Flagged or impossible values are treated as missing. |
| A6 | Each gauge's zero point is fixed, or the authority gives the offsets when it changes. | Levels can be compared across years. Each station is scaled by its own mean and spread. |
| A7 | No backwater: within one hour, a downstream level does not push an upstream level up. | A station may only attend to itself, nodes upstream of it, the rain gauges and the reservoir. |
| A8 | Travel time from the reservoir to S4 is under 12 hours at all flows. It is shorter at high flow. | A 24-hour history window covers every travel time with margin. Attention picks the lag, so no fixed lag is assumed. |
| A9 | R2 lies close to the catchment of the unmonitored tributary (as in the picture). | R2 rain acts as a stand-in for the unmonitored inflow into S3 and S4. |
| A10 | Release schedules are normally announced at least one hour before they take effect. A newer announcement replaces older ones. Emergency changes can come with less notice. | The planned release for $t_0 + 10$ to $t_0 + 60$ min is a known future input. |
| A11 | Recorded actual releases arrive within 10 minutes. | Past actual releases are a normal history input. |
| A12 | No rain forecast is available. Rain falling after $t_0$ can raise levels within the hour only on small, fast catchments. | This is not modelled directly. It is covered by the quantile band (Section 6). |
| A13 | The river layout does not change, and water taken out or evaporated within one hour is small. | The connection mask and the distances are fixed inputs. |

## 3. Forecasting task and data specification

- **Issuance time** $t_0$: every 10 minutes, on the grid (for example 08:00, 08:10, ...).
- **History window**: the last 24 hours, steps $k = -143, \ldots, 0$ at times $t_0 + 10k$ minutes (A8).
- **Horizon**: one hour, that is 6 steps. We forecast only the level at $t_0 + 60$ min.
- **Outputs**: for each of S1–S4, the 10%, 50% and 90% quantiles of the level at $t_0 + 60$. The 50% value is the main forecast.

| Name | Shape | Axes | Time coverage |
|---|---|---|---|
| Level values $X^{\text{lev}}$ | $[B, 144, 5]$ | batch, 10-min step, station (S1, S2, S3, S4, T1); metres, scaled per station | $t_0 - 1430 \ldots t_0$ min |
| Level mask, age | $[B, 144, 5]$ each | same; mask = 1 if a reading is present; age = minutes since the last valid reading | same |
| Rain $X^{\text{rain}}$ | $[B, 144, 2]$ | batch, step, gauge (R1, R2); mm in the 10 min ending at the step | same |
| Rain mask, age | $[B, 144, 2]$ each | same | same |
| Release, actual $Q^{\text{act}}$ | $[B, 144, 1]$ | batch, step, reservoir; $m^3/s$ | same |
| Release, past plan $Q^{\text{plan}}$ | $[B, 144, 1]$ | batch, step, reservoir; the plan that was in force for that step | same |
| Release, future plan $Q^{\text{fut}}$ | $[B, 6, 1]$ | batch, future step, reservoir; latest schedule announced at or before $t_0$, plus minutes since announcement | $t_0 + 10 \ldots t_0 + 60$ min |
| Clock features $\tau$ | $[B, 150, 4]$ | batch, step, (sin and cos of time of day, sin and cos of day of year) | $t_0 - 1430 \ldots$ $t_0 + 60$ min |
| Static node features $G$ | $[8, 5]$ | node, (river km from reservoir, node type one-hot $\times 4$); no batch axis, shared by all samples | fixed |
| Connection mask $U$ | $[8, 8]$ | query node, key node; 1 = may attend | fixed |
| **Output** $\hat{Y}$ | $[B, 4, 3]$ | batch, station (S1–S4), quantile (10, 50, 90%) | $t_0 + 60$ min |
| Training target $Y$, mask | $[B, 4]$ each | batch, station; level measured at $t_0 + 60$, mask = 1 if it exists | $t_0 + 60$ min |

**Alignment.** All series use the same 10-minute grid ending at $t_0$. Levels go to the slot with the same timestamp (A1). Rain is summed per slot (A3). Releases are a step function read at each slot. Places are lined up on the node axis. Because stations are unevenly spaced, the model does not treat "next index" as "next to each other on the river". The map enters through the river distances in $G$ and the connection mask $U$.

**Available at issuance.** Only values that have *arrived* by $t_0$: past levels, past rain, past releases, and schedules announced by $t_0$. Not available: future rain, future levels, future actual releases, and any flow from the unmonitored tributary.

## 4. Model computations and design justification

**Notation.** $n$ is a node (one of 8), $s$ is a station (S1–S4), $k$ is a history step, $d = 64$. $x_{k,n}$ is the raw feature vector of node $n$ at step $k$ (value, mask, age, and for the reservoir also the planned release and plan minus actual).

**Step 1 – projection.** Each source type $c(n)$ (level, rain or release) has its own weights:

$$z^{(0)}_{k,n} = W_{c(n)}\, x_{k,n} + e_n + w_k, \qquad e_n = \text{MLP}(g_n), \qquad w_k = W_\tau \tau_k.$$

Here $e_n$ comes from the node's map features $g_n$ and $w_k$ from the clock features. The six future plan tokens are $p_j = W_p\,[q^{\text{fut}}_j, a_j] + e_{\text{Res}} + w_j$, $j = 1, \ldots, 6$.

**Step 2 – temporal encoder** (3 causal T blocks, dilation $\delta \in \{1, 2, 4\}$, same weights for every node):

$$u_{k,n} = \sum_{i=0}^{2} A_i \, z_{k-i\delta,\, n} + b, \qquad z_{k,n} \leftarrow z_{k,n} + \mathrm{GELU}\big(\mathrm{LN}(u_{k,n})\big).$$

Its reach is $2(1 + 2 + 4) = 14$ steps (140 min). This captures short trends such as how fast a level is rising.

**Step 3 – space–time attention** (2 blocks, 4 heads). All $L = 8 \cdot 144 + 6 = 1158$ tokens $h_\ell$ attend to each other:

$$\alpha_{\ell m} = \mathrm{softmax}_m\!\left(\frac{q_\ell \cdot k_m}{\sqrt{d_h}} + \beta_{n(\ell),\, n(m)} + \mu_{\ell m}\right), \qquad \mu_{\ell m} = \begin{cases} 0 & \text{if } n(m) \in U(n(\ell)) \\ -\infty & \text{otherwise} \end{cases}$$

$$h_\ell \leftarrow h_\ell + W_o \sum_m \alpha_{\ell m} v_m, \qquad h_\ell \leftarrow h_\ell + \mathrm{FFN}\big(\mathrm{LN}(h_\ell)\big).$$

$\beta$ is a learned bias for each pair of nodes and each head. $U(n)$ holds $n$ itself, the stations and T1 upstream of $n$, the reservoir and both rain gauges. For example, $U(\mathrm{S1}) = \{\mathrm{S1}, \mathrm{Res}, \mathrm{R1}, \mathrm{R2}\}$ and $U(\mathrm{S4})$ = all nodes.

**Step 4 – forecast.** Station $s$ reads its own token at $k = 0$, called $r_s$:

$$(a, b, c) = \mathrm{MLP}_s(r_s), \qquad \Delta^{0.5} = a, \qquad \Delta^{0.1} = a - \mathrm{softplus}(b), \qquad \Delta^{0.9} = a + \mathrm{softplus}(c),$$

$$\hat{y}_s^{(q)} = y_s^{\mathrm{last}} + \sigma_s \, \Delta^{(q)}.$$

$y_s^{\mathrm{last}}$ is the latest level that has arrived and $\sigma_s$ is the station's spread. The softplus terms keep the three quantiles in the right order.

**Training objective** – pinball (quantile) loss, skipping missing targets:

$$\mathcal{L} = \frac{1}{\sum m_{b,s}} \sum_{b,s} m_{b,s} \sum_{q \in \{0.1, 0.5, 0.9\}} \rho_q\left(\frac{y_{b,s} - \hat{y}_{b,s}^{(q)}}{\sigma_s}\right), \qquad \rho_q(u) = \max\big(qu,\ (q-1)u\big).$$

We train with Adam and use a later time period for validation (A4).

**Why these choices.**

- **Spatial path.** A model that only sees a station's own past cannot know that a wave is coming from upstream. The model needs paths from upstream stations, T1, the rain gauges and the reservoir.
- **Attention instead of stacked local blocks.** A local space–time stack only sees as far as its depth allows. The reservoir is 4 hops above S4, and travel times can be many 10-minute steps that change with flow. A local stack would need to be very deep, and the right lag would have to be fixed in advance. With attention, each station's latest token links directly to any allowed reading in the 24 hours, and the weights depend on the input, so the lag can change with conditions. Attention costs grow with the square of the token count, but here $L = 1158$ is small (Section 5).
- **T blocks first.** They cheaply turn raw readings into local trend features before attention.
- **Upstream mask.** It follows the physics (A7). It also stops links that are only chance patterns in the data, and it lowers the amount of data needed.
- **Predicting the change.** "Same level as now" is a strong baseline for 1 hour ahead. Predicting the change keeps the output near that baseline and copes better with levels outside the training range.
- **Quantile loss.** When information is missing (the unmonitored tributary, future rain), the same inputs can lead to different outcomes. An absolute-error loss gives the median (our 50% value), which is not pulled by rare extremes. The 10% and 90% values show how uncertain the forecast is.

**How each source reaches each forecast.**

- **Upstream conditions.** S2–S4 attend to upstream station tokens; the learned weights pick the lag that matches the current travel time.
- **Tributaries.** S2–S4 attend to T1. The unmonitored tributary has no data. Its effect is learned from R2 rain (A9) and from S3 readings that cannot be explained by S2 alone.
- **Rainfall.** All stations attend to R1 and R2 over the last 24 hours, which covers delayed runoff.
- **Reservoir releases.** All stations attend to past actual releases. The future plan tokens matter most for S1, which is closest. Stations further down are affected only when the travel time is under one hour.

| **Physical assumptions (fixed by us)** | **Learned from data** |
|---|---|
| River connections and flow direction (mask $U$); river distances (node features); time order: only data that had arrived by $t_0$, plus announced releases; forecast = current level + change. | Travel times and how they change with flow (attention weights over lags); how waves flatten as they travel; rain-to-river response; the unmonitored tributary's effect; how far real releases differ from the plan. |

## 5. Operation and exceptional conditions

**(i) Missing observations.** A missing value is filled with the last valid value of that node, its mask is set to 0 and its age keeps growing. The model sees that the value is not fresh. During training we hide random single readings and random blocks of up to 2 hours, so the model learns to rely more on neighbours and rain when a station is silent. If a target is missing, it is skipped in the loss. If a station's own recent level is missing, $y_s^{\text{last}}$ is its latest valid reading, and the age input tells the model how old that reading is.

**(ii) Delayed observations.** The forecast uses only what has arrived by $t_0$. A late reading counts as missing for now (mask 0, age growing), and the next run 10 minutes later will include it. Training uses the same "as-of" rule (A2), so the model learns from the same delays it will meet in use.

**(iii) Announced future releases.** The latest schedule announced at or before $t_0$ gives 6 future tokens ($t_0+10 \ldots t_0+60$). Schedules announced after $t_0$ are not used, even in training. The "minutes since announcement" input shows how fresh the plan is.

**(iv) Scheduled vs actual releases.** In training, the future input is the plan *as it was known at* $t_0$, never the later actual release. The model therefore learns how far plans usually differ from what happens. The past window holds both the plan and the actual release for each step, so a recent gap between them is visible to the model. Once a different actual release is recorded, it enters the next forecast within 10 minutes (A11).

**Running every ten minutes.** For each run we read the last 24 hours of 8 series (a few thousand numbers) and run one forward pass. Leading cost: T blocks $\approx 3 \cdot 1152 \cdot 3d^2 \approx 4 \times 10^7$ operations; attention $C_A = 2L^2 d \approx 2 \cdot 1158^2 \cdot 64 \approx 1.7 \times 10^8$ per block, $\approx 3.4 \times 10^8$ for 2 blocks. In total this is under one billion operations, which takes well under one second on an ordinary CPU. A 10-minute cycle leaves a very large margin.

## 6. Uncertainty, limitations, and additional information

| **Issue** | **Effect on the forecast** | **Our response / what remains unresolved** |
|---|---|---|
| Unmonitored tributary | Sudden inflow raises S3 and S4 without any upstream sign. Forecasts at S3 and S4 may be too low. | R2 rain is used as a stand-in (A9), and the quantile band widens after rain near R2. **Unresolved:** a storm that misses R2 but hits the tributary. |

| **Issue** | **Effect on the forecast** | **Our response / what remains unresolved** |
|---|---|---|
| Changing travel times | The wave arrives earlier at high flow, so a fixed lag would be wrong. | Attention picks the lag from current conditions. **Unresolved:** floods bigger than any in the record may travel in ways the model never saw. |
| Rain after $t_0$ | On fast catchments, new rain can raise levels within the hour. | This shows up only as forecast spread. A rain nowcast would help (below). |
| Rare extreme floods | Few training examples, so errors are largest when they matter most. | Predicting the change limits the damage. Flood periods are checked on their own in validation. **Unresolved:** record-breaking events. |
| Plan vs actual releases | Unannounced changes cause errors at S1 first, then further down. | Learned typical gaps; the forecast is corrected within 10 minutes once the actual release is recorded. |
| Missing or faulty sensors | Long outages remove a direct path. | Masks, ages and missing-data training. Forecast spread grows. Values that do not change at all for a long time are flagged. |
| Backwater ignored (A7) | Near tributary joins, a high downstream level could raise upstream levels. | Accepted as a limit. Can be removed by allowing downstream attention if errors show it matters. |
| Quantile calibration | The 10–90% band may be too narrow or too wide. | Checked on the held-out test period: about 80% of outcomes should fall inside the band. |

**Additional information requests.**

| **Request** | **How it helps** | **Usable without it?** |
|---|---|---|
| Short-term rain forecast (nowcast or radar) for the next hour | Covers rain that falls after $t_0$; added as future rain tokens like the release plan. | Yes |
| A gauge (even temporary) on the unmonitored tributary | Turns the biggest unknown into a measured input for S3 and S4. | Yes |
| Travel-time studies, river cross-sections or rating curves | Lets us check that the learned lags make physical sense, and convert levels to flows. | Yes |
| Live actual-release data and reservoir operating rules | Faster correction when releases differ from the plan. | Yes |
| Sensor metadata: delay logs, quality flags, zero-point changes | Confirms A2, A5 and A6 and improves the as-of training data. | Yes, using the stated assumptions |
| Record length and list of past floods | Confirms A4. If the record is short, the model must be made smaller. | Yes, but quality depends on it |

# Question 2 – Motion or Memory?

Code: `Question2/` in the repository. Every number below uses the 1,000 test videos (9,000–9,999) unless stated. Learned models are shown as *mean (seed 0, seed 1)*. Figures use the seed-0 models. Hardware: one NVIDIA GeForce RTX 3070 Ti Laptop GPU (8 GB), PyTorch 2.11.0 (CUDA 12.8).

## 1. Setup

Table 1: Checks of the transformation and window functions (synthetic tensors).

| | Check | Expected | Yours |
|---|---|---|---|
| (a) | $\max \lvert M(M(x)) - x \rvert$ | 0 | 0 (`torch.equal`: True) |
| (b) | $\max \lvert R^4(x) - x \rvert$ | 0 | 0 (`torch.equal`: True) |
| (c) | $\max \lvert MRM(x) - R^{-1}(x) \rvert$ | 0 | 0 (`torch.equal`: True) |
| (d) | bright pixel of $M(p)$ (frame, row, column) | (7, 10, 43) | (7, 10, 43) |
| (e) | bright pixel of $R(p)$ (frame, row, column) | (7, 43, 10) | (7, 43, 10) |
| (f) | 1× input / target frames | 1–5 / 6–10 | 1, 2, 3, 4, 5 / 6, 7, 8, 9, 10 |
| (g) | 2× input / target frames | 1, 3, 5, 7, 9 / 11, 13, 15, 17, 19 | 1, 3, 5, 7, 9 / 11, 13, 15, 17, 19 |
| (h) | 3× input / target frames | 1, 4, 7, 10, 13 / 16, 19 | 1, 4, 7, 10, 13 / 16, 19 |

All eight checks match exactly.

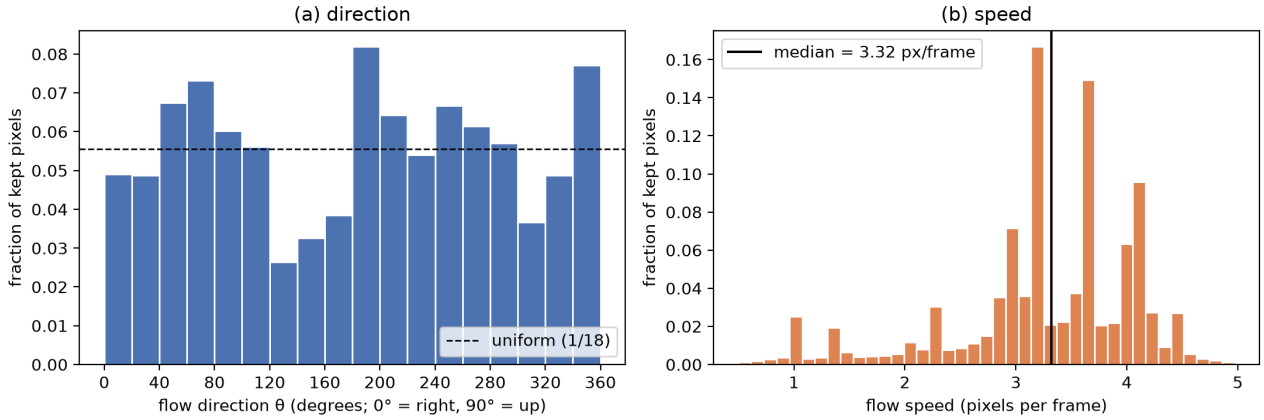



Figure 1: Optical flow on the first 50 training videos (950 frame pairs, Farneback). (a) Direction of motion, 18 bins of 20°; the dashed line is a uniform spread (1/18). (b) Speed in pixels per frame. Both show the fraction of kept pixels. Median speed: 3.32 pixels per frame. Pixels kept: 103,050.

**Q2.1** Yes. Mirrored and rotated motion is already familiar. In Figure 1(a) every 20° direction holds between 2.6% and 8.2% of the moving pixels; no direction is missing. Mirroring turns a direction $\theta$ into $180° - \theta$, and a 90° rotation turns it into $\theta + 90°$. Both only reshuffle the bars of a histogram that already covers every direction, so a mirrored or rotated video moves in directions the model has seen many times. The bars are uneven only because 50 videos contain about 100 digits, and each digit keeps one direction between bounces. Figure 1(b) shows a typical speed of 3.32 pixels per frame (half of all pixels between 3.00 and 3.89). So 2× is about 6.6 and 3× about 10.0 pixels per frame, both faster than almost anything in training. Where digits overlap, the flow is wrong, which likely explains the small bumps near 1 pixel per frame. Overlaps are short and happen in all directions, so they add a little noise but cannot fill or empty a whole direction. The answer does not change.

**Q2.2** Mirror and rotation change only the *appearance* of the digits; the motion statistics stay the same, because the direction histogram is already full (Q2.1) and the speed does not change. Rotation

changes appearance more than mirroring. Speed-up does the opposite: it changes only the *motion* (each step is $k$ times longer), while the digits look the same. So each test changes one thing and keeps the other fixed. If the model fails a mirror or rotation test, the cause is appearance. If it fails a speed test, the cause is motion. This is what makes them a controlled experiment.

## 2. Baseline: ConvLSTM

Table 2: Model A against the two reference predictors on the original test clips ($1\times$, $t = 1$).

| Measure | Model A | Blank | Copy-last |
|---|---|---|---|
| $\mathrm{MSE}_1$ | 0.0170 (0.0175, 0.0165) | 0.0426 | 0.0492 |
| $\mathrm{MSE}_2$ | 0.0204 (0.0209, 0.0198) | 0.0426 | 0.0604 |
| $\mathrm{MSE}_3$ | 0.0226 (0.0231, 0.0220) | 0.0426 | 0.0669 |
| $\mathrm{MSE}_4$ | 0.0243 (0.0248, 0.0238) | 0.0425 | 0.0708 |
| $\mathrm{MSE}_5$ | 0.0259 (0.0263, 0.0256) | 0.0425 | 0.0732 |
| $E^{(5)}$ | 0.0220 (0.0225, 0.0215) | 0.0426 | 0.0641 |
| Trainable parameters | 480,784 | 0 | 0 |




Figure 2: Model A (seed 0) on three test videos: input frames 1–5, predicted frames 6–10, and the true frames 6–10 below. In video 9000 the two digits overlap.

**Q2.3** Model A's reach is **8 pixels per time step**. One cell of the folded $16 \times 16$ grid is 4 original pixels. Pixel-unshuffle, the $1 \times 1$ output convolution and pixel-shuffle do not move information. Only the two $3 \times 3$ convolutions do, and each moves it by at most one cell (4 pixels). Within one step $\tau$:

- from the new input $u_\tau$: layer-1 convolution (1 cell) $\to h_\tau^{(1)} \to$ layer-2 convolution (1 cell) $\to h_\tau^{(2)} \to$ output: **2 cells**;
- from $h_{\tau-1}^{(1)}$: it enters the same layer-1 convolution, so the path is also **2 cells**;
- from $h_{\tau-1}^{(2)}$: only the layer-2 convolution, **1 cell**;

- the cell states $c_{\tau-1}^{(\ell)}$ are only multiplied pixel by pixel, so they add no distance (at most the 1 cell of layer 2 for $c^{(1)}$).

The longest path is 2 cells $= 2 \times 4 = 8$ pixels. A digit that moves more than about 8 pixels between frames cannot be linked to its old position in a single update.

**Q2.4** Model A is much better than both reference predictors: its $E^{(5)}$ is 0.0220, against 0.0426 for Blank and 0.0641 for Copy-last (Table 2). Copy-last is the worst because a copied digit is wrong twice: it is missing where the digit went and present where the digit was. Model A's error grows steadily over the five horizons, from 0.0170 at $h = 1$ to 0.0259 at $h = 5$ (52% higher), but even at $h = 5$ it stays far below Blank. Blank does not change with $h$, while Copy-last grows faster than model A (0.0492 to 0.0732) because the digits move further from their last position. Figure 2 shows why model A's error grows: each prediction is fed back as the next input, so small errors add up and the digits get blurrier with every step. They stay in roughly the right places, but by frame 10 they are grey smudges. When digits overlap (video 9000), the model cannot tell which strokes belong to which digit. It predicts one merged smear that drifts upward, and the two digits never separate, even though the true 5 and 8 move apart.

**Q2.5** (written and committed before any experiment in Testing the Baseline; copied unedited from `Question2/predictions/Q2.5.md`)

> **Mirror test.** Figure 1(a) shows digits moving in every direction, so mirrored motion is motion model A has already seen. Only the look of the digits is new (a mirrored 2 is not a 2). I expect model A to mostly pass: the digits should still move the right way, but their shapes may blur or drift back towards normal-looking digits. So E_M should be a little higher than E(5), and C_M small but not zero.
>
> **Rotation test.** The motion is again familiar, but rotated digits look even less like the training digits than mirrored ones. I expect model A to do worse than on the mirror test (C_R > C_M and E_R > E_M), while the digits still move in roughly the right direction.
>
> **Speed.** The median speed is about 3.3 pixels per frame, so 2x is about 6.6 and 3x about 10 pixels per frame. Model A can move information at most 8 pixels per step. At 2x the motion is still inside this reach, but the model never saw such fast motion in training, so I expect some extra error but no breakdown. At 3x most motion is beyond the reach, so I expect model A to break down there: blurred digits that stay near their old position or fade away.

## 3. Testing the Baseline

Table 3: Consistency gaps and transformed errors at 1×, $t = 1$. $C_T = \mathrm{MSE}(f(T(X)), T(f(X)))$, $E_T = \mathrm{MSE}(f(T(X)), T(Y))$.

| Measure | Model A | Blank | Copy-last |
|---|---|---|---|
| $E^{(5)}$ | 0.0220 (0.0225, 0.0215) | 0.0426 | 0.0641 |
| $C_M$ | 0.00181 (0.00162, 0.00200) | 0 | 0 |
| $E_M$ | 0.0227 (0.0231, 0.0223) | 0.0426 | 0.0641 |
| $C_R$ | 0.00308 (0.00275, 0.00341) | 0 | 0 |
| $E_R$ | 0.0238 (0.0241, 0.0235) | 0.0426 | 0.0641 |

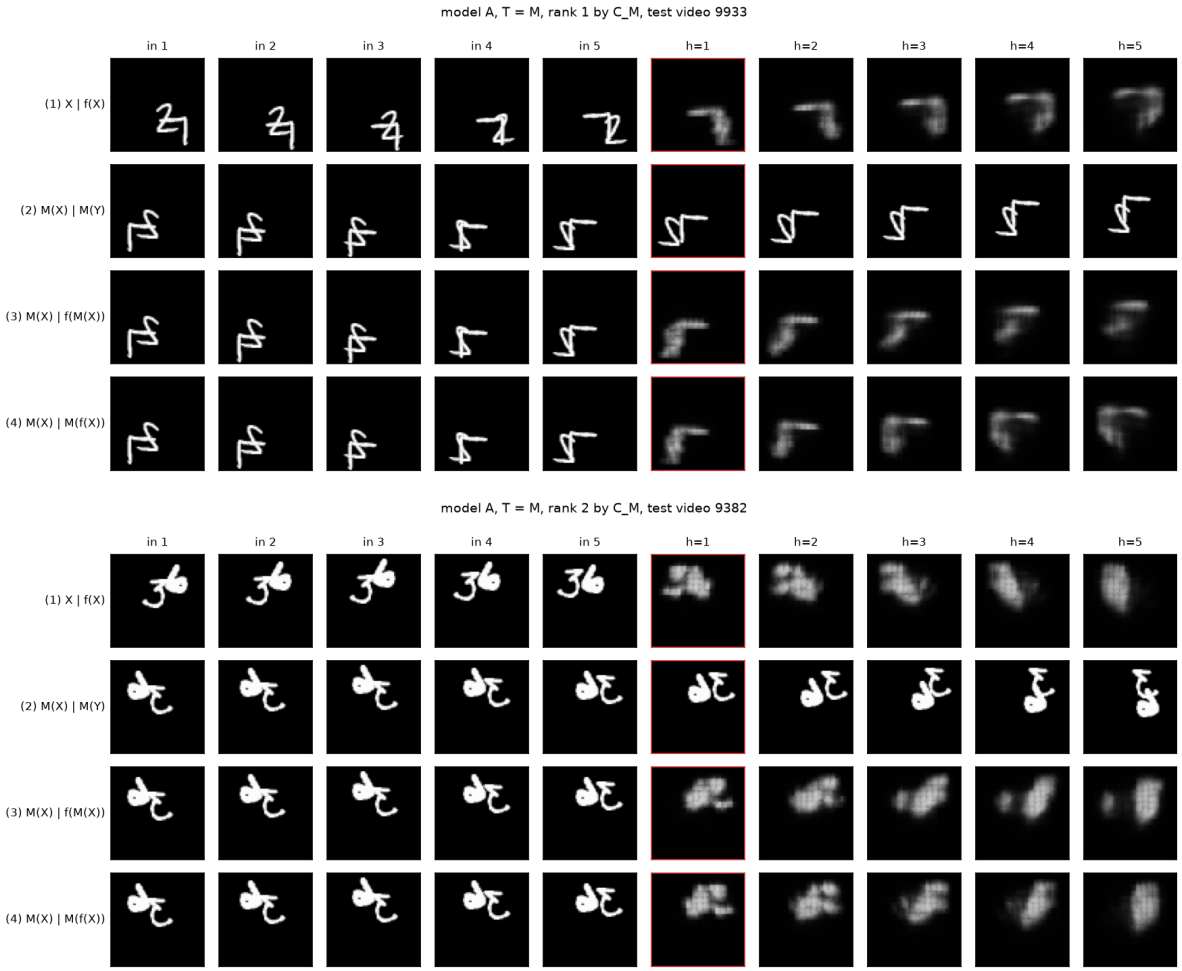



Figure 3: Model A (seed 0): the two test videos with the largest per-video $C_M$ (videos 9933 and 9382). Rows: (1) $X \mid f(X)$, (2) $M(X) \mid M(Y)$, (3) $M(X) \mid f(M(X))$, (4) $M(X) \mid M(f(X))$. GIFs: `Question2/gifs/figure3_M1_video9933.gif`, `Question2/gifs/figure3_M2_video9382.gif`.

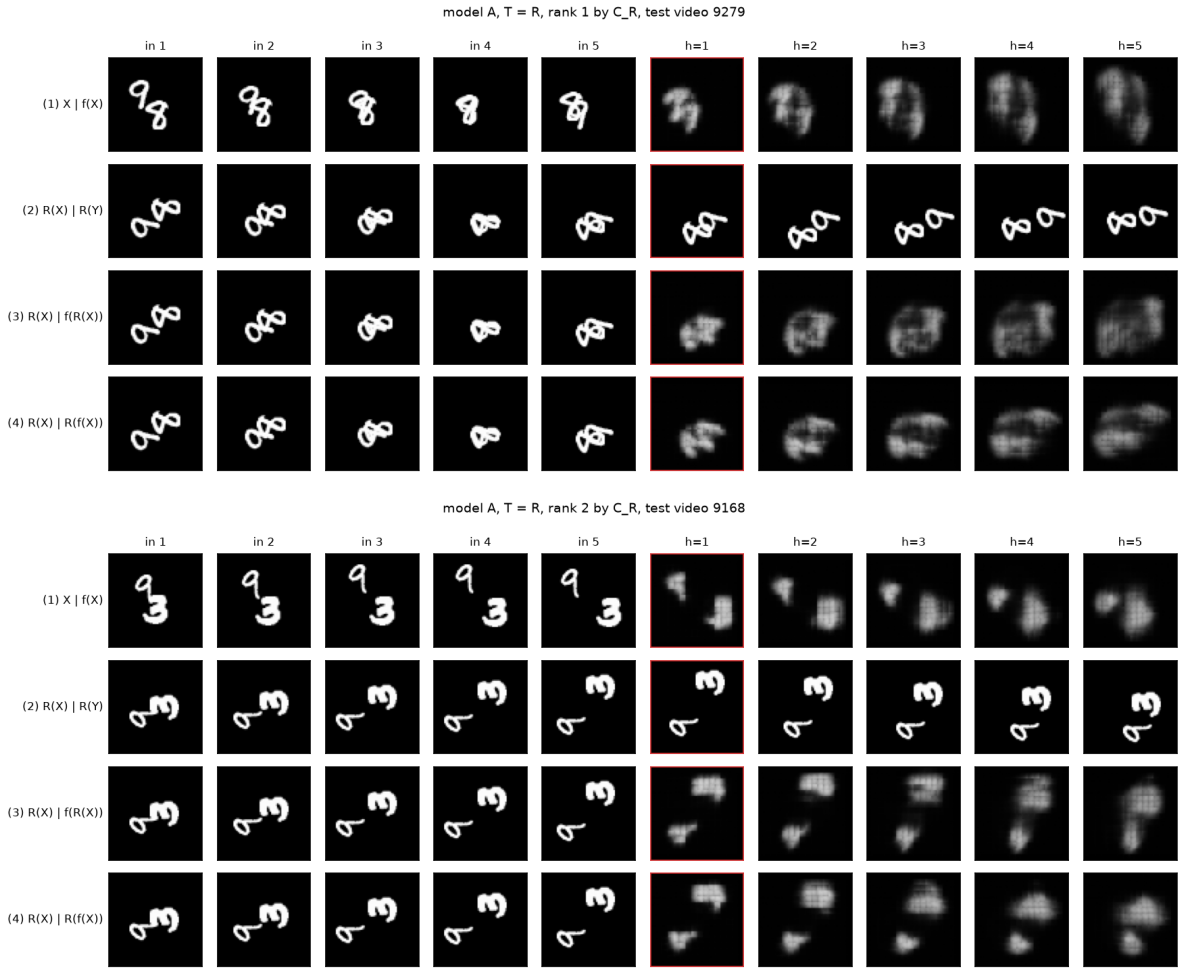



Figure 3: (continued) The two test videos with the largest per-video $C_R$ (videos 9279 and 9168). Rows as above with $T = R$. GIFs: `Question2/gifs/figure3_R1_video9279.gif`, `Question2/gifs/figure3_R2_video9168.gif`.

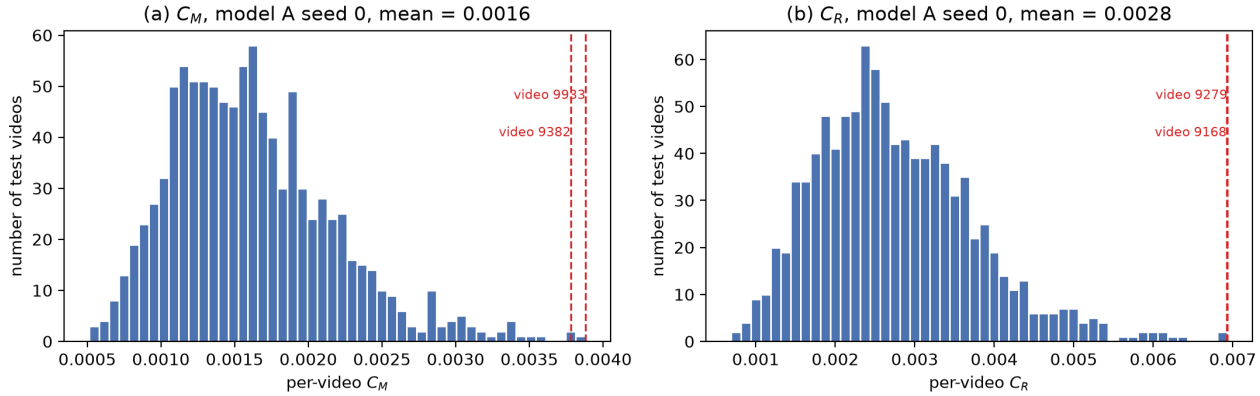



Figure 4: Per-video $C_M$ (a) and $C_R$ (b) of model A (seed 0) over the 1,000 test videos. Dashed red lines mark the Figure 3 videos.

**Q2.6** A model can be perfectly consistent and still useless. Blank and Copy-last have $C_M = C_R = 0$ exactly, because they never look at the motion: a blank frame stays blank and a copied frame is just the input moved around. Yet their transformed errors ($E_M = E_R = 0.0426$ and 0.0641) are about two and three times model A's. A small $C_T$ only says that the model treats the transformed video the same way as the original. It does not say whether that treatment is correct. That is why $E_T$ must

always be reported next to it.

**Q2.7** Video 9279 under rotation (Figure 3, third panel). In the original video (row 1), a 9 and an 8 overlap, and model A predicts a blob that stretches up and down, which is roughly the direction the 9 is moving. Rotated, the true future (row 2) is the 9 moving to the right, away from the 8. Rotating the original prediction (row 4) gives a blob that stretches left and right, which is the right direction. But when the model sees the rotated video itself (row 3), it loses the direction completely: it predicts a round cloud that grows equally in every direction and gets a grid-like texture. The 9 never separates from the 8 and never moves right, and by $h = 5$ neither digit can be recognised. So on the rotated video the model no longer knows which way to move the digits, and it blurs everything in place.

**Q2.8** Yes, rotation is harder than mirroring for model A: $C_R = 0.00308$ against $C_M = 0.00181$, and $E_R = 0.0238$ against $E_M = 0.0227$ (both seeds agree; Table 3). The difference cannot come from the motion. Q2.1 showed that both transformations only move digits in directions that are already common in training, at the same speed. So the cause must be appearance (Q2.2). A mirrored digit keeps up as up, its strokes keep their direction, and many digits (0, 1, 8) look almost the same mirrored. A rotated digit has its vertical strokes turned horizontal, so it looks less like any digit the model has seen. The more unfamiliar the digits look, the worse the model follows them.

Table 4: Model A at faster playback ($t = 1$). Across speeds only the first two horizons are compared. Blank for comparison: $E^{(2)} = 0.0426$, 0.0425, 0.0426 at 1×, 2×, 3×.

| Measure | Model A | Copy-last |
|---|---|---|
| $E^{(2)}$ at 1× | 0.0187 (0.0192, 0.0181) | 0.0548 |
| $E^{(2)}$ at 2× | 0.0451 (0.0441, 0.0462) | 0.0661 |
| $E^{(2)}$ at 3× | 0.0531 (0.0526, 0.0537) | 0.0703 |
| $E^{(5)}$ at 1× | 0.0220 (0.0225, 0.0215) | 0.0641 |
| $E^{(5)}$ at 2× | 0.0503 (0.0491, 0.0516) | 0.0719 |

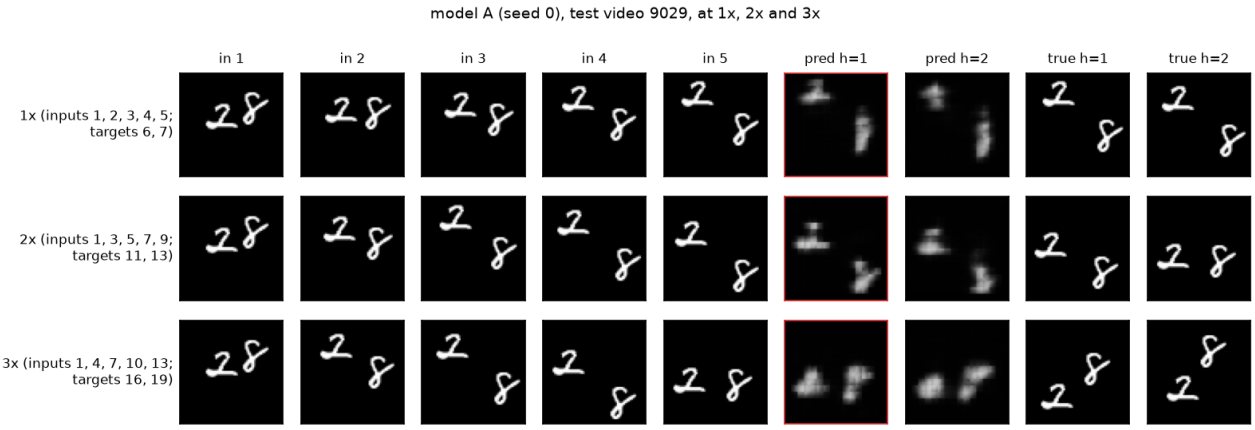



Figure 5: Model A (seed 0), test video 9029 at 1×, 2× and 3×: five input frames, the first two predicted frames and the first two true frames. The digits do not overlap in any shown true frame.

**Q2.9** Model A falls apart already at **2**×. $E^{(2)}$ jumps from 0.0187 at 1× to 0.0451 at 2×, which is worse than predicting a blank frame (0.0425), and 0.0531 at 3× (Table 4). This does not agree with my reach estimate, which said 2× (about 6.6 pixels per frame) is still inside the 8-pixel reach. The estimate missed two things. First, reach only says a path *exists*. The paths near the edge of two stacked $3 \times 3$ kernels pass through few weights, so the distance the network can use well is shorter than 8 pixels. Second, a path is useless if the model never learned to use it: model A only ever saw digits move about 3.3 pixels per frame. In Figure 5, at 2× the predicted 2 stalls near its last input position while the true 2 moves on, and the 8 loses its shape. At 3× both digits stay where the last

input frame left them and turn into grey blobs, while the true digits are far away. Stalling means the model cannot connect the old and new positions, so it keeps the digit where it last saw it. Blurring means it is unsure where the digit went, and the squared-error loss rewards spreading the guess over the possible places.

**Q2.10** No. Model A was trained only at 1×, so at 2× and 3× the motion is both unfamiliar and close to or beyond its reach, and these results cannot separate the two causes. Experiment: train the same architecture with speed augmentation (model C), so fast motion is no longer unfamiliar. If C fixes a speed, that failure was unfamiliar motion. If C still fails at a speed while a model with a larger reach, trained the same way (model D), succeeds, that failure was the architecture's reach.

## 4. What Can Data Fix?

Models B, C and D follow the handout's model table and the same training protocol as model A. B adds symmetry augmentation, C adds speed augmentation, and D is my larger-reach design trained like C (described in Q2.12). D+B was not trained (see Section 5).

**Q2.11** (written and committed before training B–D; copied unedited from `Question2/predictions/Q2.11.md`)

> Model B (mirror and rotation augmentation) will lower C_M and C_R and bring E_M and E_R close to E(5), because those failures come from the digits' appearance, but it will not help at 2x or 3x. Model C (speed augmentation) will fix much of the 2x failure, because 2x motion (about 6.6 pixels per frame) is inside model A's 8-pixel reach and was only unfamiliar, but it will still do badly at 3x (about 10 pixels per frame). Only model D, with a 12-pixel reach and speed augmentation, will also fix 3x, while neither C nor D will change C_M or C_R much.

Table 5: Models A–D on the test set ($t = 1$). Training time is the mean over 30 epochs; inference time covers all 1,000 original test videos at batch size 32.

| Measure | Model A | Model B | Model C | Model D |
|---|---|---|---|---|
| $E^{(5)}$ at 1× | 0.0220 (0.0225, 0.0215) | 0.0228 (0.0233, 0.0223) | 0.0274 (0.0285, 0.0263) | 0.0276 (0.0281, 0.0271) |
| $C_M$ | 0.00181 (0.00162, 0.00200) | 0.00116 (0.00119, 0.00112) | 0.00152 (0.00186, 0.00118) | $6.60 \times 10^{-4}$ ($5.89 \times 10^{-4}$, $7.30 \times 10^{-4}$) |
| $E_M$ | 0.0227 (0.0231, 0.0223) | 0.0228 (0.0233, 0.0223) | 0.0277 (0.0288, 0.0266) | 0.0278 (0.0283, 0.0273) |
| $C_R$ | 0.00308 (0.00275, 0.00341) | 0.00146 (0.00155, 0.00137) | 0.00256 (0.00272, 0.00240) | 0.00164 (0.00158, 0.00169) |
| $E_R$ | 0.0238 (0.0241, 0.0235) | 0.0228 (0.0233, 0.0222) | 0.0285 (0.0295, 0.0276) | 0.0286 (0.0290, 0.0282) |
| $E^{(2)}$ at 1× | 0.0187 (0.0192, 0.0181) | 0.0192 (0.0197, 0.0187) | 0.0235 (0.0244, 0.0227) | 0.0248 (0.0252, 0.0243) |
| $E^{(2)}$ at 2× | 0.0451 (0.0441, 0.0462) | 0.0435 (0.0437, 0.0433) | 0.0291 (0.0295, 0.0288) | 0.0291 (0.0294, 0.0287) |
| $E^{(2)}$ at 3× | 0.0531 (0.0526, 0.0537) | 0.0498 (0.0497, 0.0499) | 0.0336 (0.0339, 0.0334) | 0.0326 (0.0328, 0.0324) |
| Trainable parameters | 480,784 | 480,784 | 480,784 | 480,784 |
| Training time per epoch (s) | 7.32 (7.26, 7.38) | 7.54 (7.50, 7.59) | 7.40 (7.40, 7.40) | 7.81 (7.80, 7.81) |
| Inference time, 1,000 videos (s) | 0.345 (0.345, 0.346) | 0.349 (0.349, 0.350) | 0.351 (0.350, 0.352) | 0.350 (0.350, 0.349) |

**Q2.12** Model D is model A with one change: the $3 \times 3$ convolution of the *second* ConvLSTM layer is dilated by 2 (padding 2). Its nine taps now sit at offsets of $-2$, 0 and $+2$ cells instead of $-1$, 0 and $+1$. Everything else is the same as model A: $4 \times 4$ patch folding, 64 hidden channels, $1 \times 1$ output convolution, the 9-step rollout without teacher forcing, and the training protocol (with speed augmentation, like C). Dilation adds no weights, so D has 480,784 parameters, well under the 721,176 limit. Reach, counted the same way as in Q2.3:

- from the new input $u_\tau$: layer 1 (1 cell) then layer 2 (2 cells) = **3 cells**;
- from $h^{(1)}_{\tau-1}$: the same two convolutions, **3 cells**;
- from $h^{(2)}_{\tau-1}$: only layer 2, **2 cells**;
- the cell states again add no distance of their own.

The longest path is $3 \times 4 =$ **12 pixels per step**, strictly more than model A's 8. This is enough for 3× motion (about 10 pixels per frame). I left layer 1 undilated so that the two layers together reach

every offset from $-3$ to $+3$ cells with no gaps; dilating both layers would skip odd offsets and miss the small moves seen at $1\times$.

**Q2.13** Labels for the failures of model A (Table 5):

- **Mirror and rotation gaps → appearance.** Model B changes only what the digits look like in training. It halves $C_R$ (0.00308 to 0.00146), lowers $C_M$ (0.00181 to 0.00116), and makes $E_M = E_R = E^{(5)} = 0.0228$, so transformed clips become as easy as the originals. Model C changes only the motion in training and leaves $C_R$ high (0.00256). D's gaps are also smaller, probably partly because its blurrier predictions differ less from each other in every case.
- **Failure at $2\times$ → unfamiliar motion.** Model C has model A's architecture and reach and only sees faster motion in training, and it cuts $E^{(2)}$ at $2\times$ from 0.0451 to 0.0291. Model D's larger reach gives exactly the same 0.0291, so reach was not the limit at $2\times$.
- **Failure at $3\times$ → mostly unfamiliar motion, with a small reach part.** C cuts $E^{(2)}$ at $3\times$ from 0.0531 to 0.0336. D lowers it further to 0.0326, and both seeds agree (0.0328 and 0.0324 against 0.0339 and 0.0334). So the extra reach helps, but only a little.

Model C did *not* do badly at $3\times$, so my Q2.11 prediction was wrong there. The reason is that a 10-pixel move is only slightly beyond the 8-pixel reach, and a digit is about 20 pixels wide. An output pixel at the digit's new position can still see the back part of the digit in the input, so the network can move it with some error. If C had still failed at $3\times$ while D succeeded, that would have shown a pure reach limit, and more training data could not fix it. Data only changes the weights on paths that already exist: if no path joins an input pixel to an output pixel within one step, no setting of the weights can make the output depend on it. Only a change to the architecture, such as a larger reach, can create that path.

**Q2.14** Yes. Speed augmentation costs accuracy on the original $1\times$ clips: $E^{(5)}$ rises from 0.0220 (A) to 0.0274 (C) and 0.0276 (D), about 25% worse. With the same 30 epochs, only about a third of the training windows are at $1\times$, and the same network must now handle three speeds. Symmetry augmentation costs much less: B's $E^{(5)}$ is 0.0228, about 4% worse than A. For C and D I think the cost is worth it, because the gains at higher speeds are larger. At $2\times$ the error falls by 35% (0.0451 to 0.0291) and at $3\times$ by 39% (0.0531 to 0.0326), where model A was worse than a blank frame. For B the trade is smaller: it halves the consistency gap at a 4% cost, but symmetrisation (Section 5) gives exact consistency with no training cost.

## 5. The Final Model

**Why the symmetrised forecaster has $C_M = C_R = 0$.** Let $T$ be $M$ or $R$, which are both elements of $G$. Then

$$\bar{f}(TX) = \tfrac{1}{8} \sum_{g \in G} g^{-1}(f(gTX)).$$

Put $h = gT$. As $g$ runs over all of $G$, so does $h$ (multiplying every element of a group by $T$ only reorders it), and $g^{-1} = Th^{-1}$. So

$$\bar{f}(TX) = \tfrac{1}{8} \sum_{h \in G} Th^{-1}(f(hX)) = T\Big(\tfrac{1}{8} \sum_{h \in G} h^{-1}(f(hX))\Big) = T(\bar{f}(X)),$$

because $T$ only moves pixels around, so it can be taken out of the average. Therefore $C_T = \mathrm{MSE}(\bar{f}(TX), T(\bar{f}(X))) = 0$ for any $f$. In my code the measured values are about $4 \times 10^{-17}$ (Table 6), which is rounding error from adding the eight predictions in a different order.

**Choice of final model: symmetrised D.** Before seeing Table 5, I set a rule: train D+B only if B clearly lowered $E_M$ or $E_R$ while keeping $1\times$ accuracy close to A. Table 5 does not support it. B left $E_M$ unchanged (0.0228 against 0.0227), lowered $E_R$ by only 4%, and was 4% worse on original clips. Symmetrisation already makes D exactly consistent, and D is by far the best model at $2\times$ and $3\times$. So the final model is model D wrapped as a symmetrised forecaster.

Table 6: Symmetrised models (the final model is symmetrised D). Training time is that of the underlying model (B or D); symmetrisation needs no training. Inference includes all eight transformations, forward passes, inverse transformations and the averaging.

| Measure | Sym. B | Sym. D |
|---|---|---|
| $E^{(5)}$ at 1× | 0.0222 (0.0227, 0.0217) | 0.0276 (0.0281, 0.0271) |
| $C_M$ | $3.97 \times 10^{-17}$ ($4.08 \times 10^{-17}$, $3.86 \times 10^{-17}$) | $3.24 \times 10^{-17}$ ($3.43 \times 10^{-17}$, $3.04 \times 10^{-17}$) |
| $E_M$ | 0.0222 (0.0227, 0.0217) | 0.0276 (0.0281, 0.0271) |
| $C_R$ | $4.05 \times 10^{-17}$ ($4.15 \times 10^{-17}$, $3.95 \times 10^{-17}$) | $3.34 \times 10^{-17}$ ($3.56 \times 10^{-17}$, $3.13 \times 10^{-17}$) |
| $E_R$ | 0.0222 (0.0227, 0.0217) | 0.0276 (0.0281, 0.0271) |
| $E^{(2)}$ at 1× | 0.0188 (0.0193, 0.0183) | 0.0248 (0.0252, 0.0244) |
| $E^{(2)}$ at 2× | 0.0427 (0.0429, 0.0425) | 0.0291 (0.0295, 0.0287) |
| $E^{(2)}$ at 3× | 0.0488 (0.0490, 0.0487) | 0.0325 (0.0327, 0.0323) |
| Trainable parameters | 480,784 | 480,784 |
| Training time per epoch (s) | 7.54 (7.50, 7.59) | 7.81 (7.80, 7.81) |
| Inference time, 1,000 videos (s) | 2.31 (2.31, 2.31) | 2.30 (2.30, 2.31) |

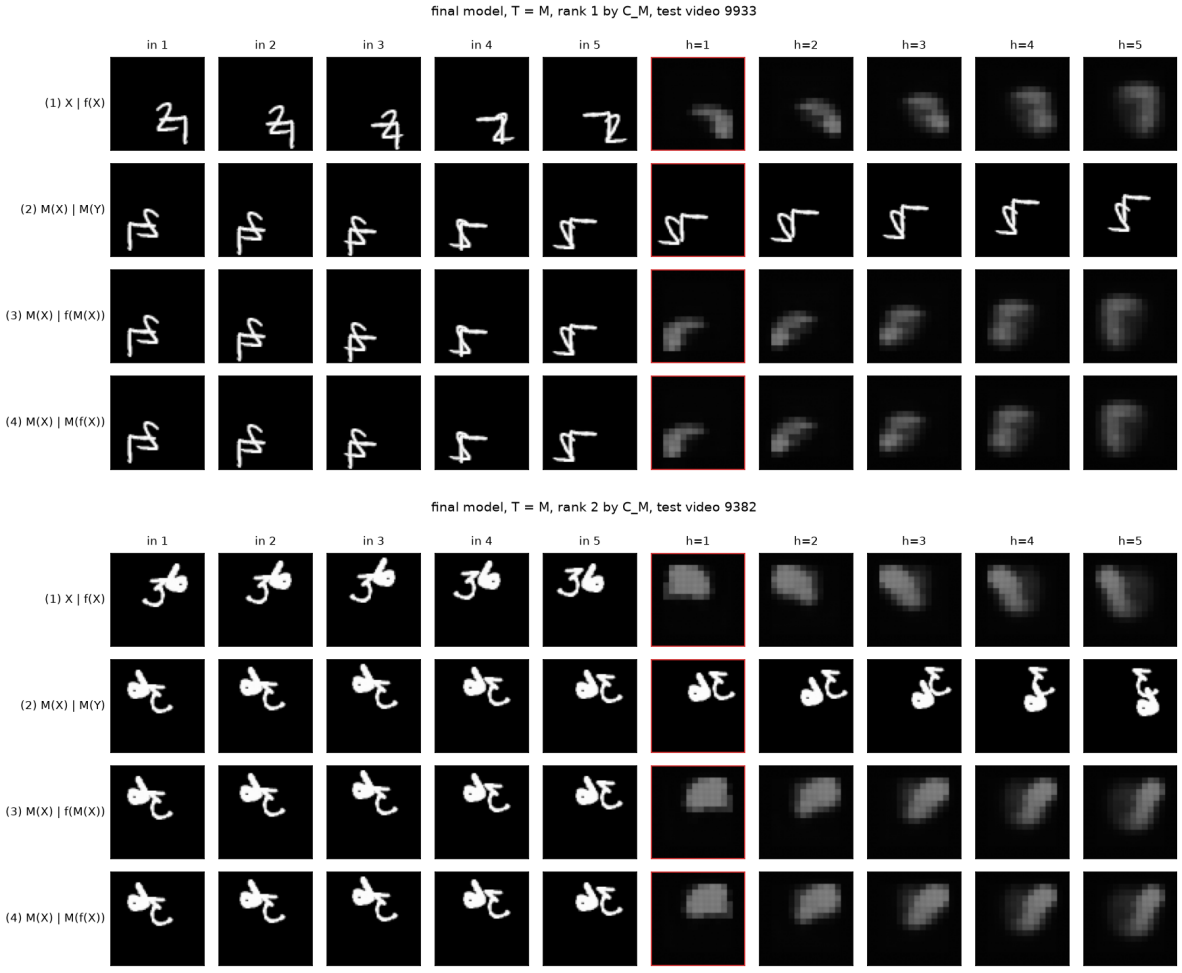



Figure 6: (a) Figure 3 repeated with the final model (symmetrised D, seed 0) on the same videos, mirror cases. GIFs: `Question2/gifs/figure6a_M1_video9933.gif`, `Question2/gifs/figure6a_M2_video9382.gif`.

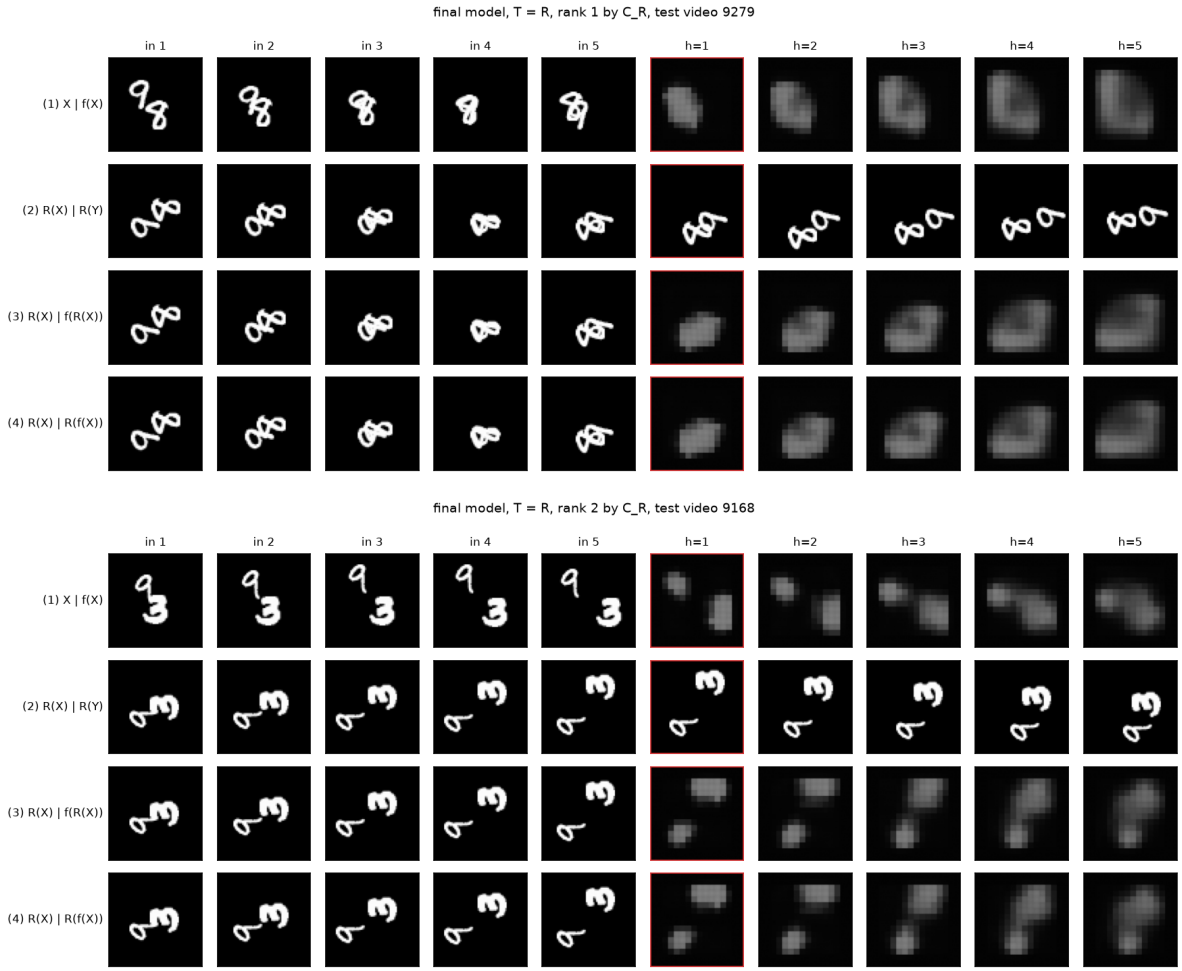



Figure 6: (a, continued) Rotation cases. GIFs: `Question2/gifs/figure6a_R1_video9279.gif`, `Question2/gifs/figure6a_R2_video9168.gif`.

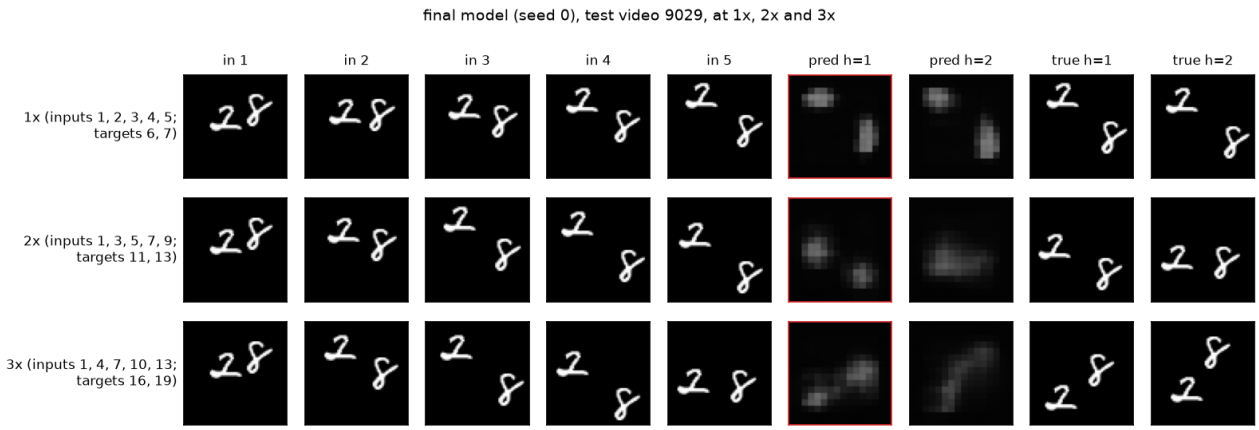



Figure 6: (b) Figure 5 repeated with the final model on the same test video (9029) at 1×, 2× and 3×.

**Q2.15** *Consistency:* learned symmetry (model B) only lowers the gaps, to $C_M = 0.00116$ and $C_R = 0.00146$. Built-in symmetry makes them zero up to rounding ($\approx 4 \times 10^{-17}$) for both B and D. Only the built-in version is exact, because its consistency follows from the proof above for *any* weights. Augmentation only rewards consistency on the training videos, so it is never guaranteed on new ones. *Accuracy:* symmetrised B is slightly better than B on original clips (0.0222 against 0.0228) and on transformed clips ($E_M = E_R = 0.0222$ against 0.0228). Symmetrised D equals D on original clips (0.0276) and is slightly better on rotated clips (0.0276 against 0.0286). *Cost:* augmentation adds about

3% to training time (7.54 against 7.32 seconds per epoch) and nothing at inference. Symmetrisation needs no training but makes inference about 6.6 times slower (2.31 against 0.35 seconds for 1,000 videos), because every input is run eight times. *Blur:* the average of eight predictions is blurry wherever they disagree. So I would expect built-in symmetry to hurt when the base model is much better on upright digits than on rotated or mirrored ones: then good and bad predictions get mixed. It did not hurt here. On original clips symmetrised B is better than B, and symmetrised D is within 0.3% of D for each seed (0.0276 for both on average), so the eight predictions agreed well enough.

**Q2.16** No, not fully. In Figure 6(a) for video 9279 (rotation), rows 3 and 4 are now identical, so the disagreement from Q2.7 is gone. Symmetrisation fixed that part. But the forecast itself is still wrong: the final model predicts one grey blob that grows over the horizons, and the 9 never separates from the 8 or moves right. The blob is even blurrier than model A's. D's larger reach and speed training did not fix this, because this failure comes from the overlapping digits (Q2.4), not from speed or appearance.

**Q2.17** Final verdict. Partly. The final model has learned a real motion rule, not just the look of the training videos. It is exactly consistent under mirroring and rotation. Its error on transformed clips equals its error on the originals. With speed training it follows digits at 2× and 3× ($E^{(2)} = 0.0291$ and 0.0325 against 0.0426 for Blank), and in Figure 6(b) its blobs land near the true positions. To convince a skeptic, I would test motion the model never saw in training, such as 4× or 1.5× speed, or letters instead of digits. If the error stays close, the model is moving shapes, not recalling digits. One test it still fails is overlap: when two digits cross (video 9279), it merges them into one growing blob instead of keeping them apart. It also blurs the digits' shapes after a few frames.

## Use of Generative AI

I used Claude (Anthropic), through Claude Code in VS Code, as a supporting tool throughout this assignment, for the following purposes:

- **Understanding the assignment.** Claude explained each question in plain English: the Question 1 design brief and its companion notebook and, for Question 2, the data split, the windowing rules for each speed and start frame, the mirror and rotation transforms and the symmetry group $G$, the consistency gap $C_T$ and transformed error $E_T$, and the reach conventions used in Q2.3 and Q2.12.
- **Question 1.** Claude helped draft the design (assumptions, data specification, architecture, training and evaluation) and drew the architecture diagram.
- **Implementation support.** Claude helped write and debug the Python code in `Question2/`. This included:
  - data loading, the train/validation/test split and the speed windows;
  - the mirror and rotation transforms;
  - the ConvLSTM models A–D and the training loop;
  - the evaluation metrics, the Blank and Copy-last baselines, the speed tests and the symmetrised forecaster;
  - the code that makes the tables, figures and GIFs.

  It also helped design model D within the parameter budget, and checked parameter counts and reach.
- **Report drafting.** Claude helped draft and edit the written answers, table layouts and figure captions from the experimental results, and fixed LaTeX layout issues such as overflowing tables and table numbering.

All experiments ran on my own computer (NVIDIA GeForce RTX 3070 Laptop GPU), where Claude Code ran the training and evaluation scripts. Every reported number comes from the code and result files in the GitHub repository. I reviewed all outputs, figures and report text before submission. The repository's commit history records that the Q2.5 and Q2.11 predictions were committed before the corresponding experiments were run, and they are reported unedited.